MyCodeSchool

Lecția 1 — Atelierul și mâna de pe cluster

pregătire · regula cursului · 4 pași · ce duci acasă · pe platforma ta din `ms\`

Înainte de lecție — ce pregătești

Nimic de instalat în afară de Claude Code. Un singur lucru de făcut: **clonează-ți cele cinci repo-uri într-un singur folder părinte**, dacă nu le ai deja așa:

```
ms/
├── data-service/
├── importer-service/
├── client-microserv-vite/
├── argo-ms-gitops/
└── ms-gitops/
```

Atât. Nu curăța nimic, nu repara nimic înainte. Ne trebuie exact în starea în care sunt azi — o să vezi imediat de ce. Adu și `kubectl` configurat pe clusterul tău, cu contextul pe el.

Ce facem azi

Îți construiești un atelier: un folder în care Claude Code știe singur cum arată platforma ta, un gard care îl oprește să pună mâna pe cluster, și prima ta unealtă adevărată — una care răspunde la o întrebare pe care ți-ai pus-o deja săptămâna trecută.

Nu învățăm „cum se scriu prompturi". Nu învățăm Kubernetes — pe ăla îl știi. Învățăm cum se transformă o verificare pe care o faci cu ochii, prin trei tab-uri de GitHub, într-o comandă pe care o rulează și un coleg, fără să fii tu lângă.

Regula pe care stă tot cursul

> **Agentul nu atinge clusterul. Agentul schimbă repo-ul. Clusterul se schimbă singur.**

Regula asta nu e despre AI. E GitOps, exact cum îl faci deja: `kubectl apply` cu mâna e un lucru pe care ArgoCD îl va anula la următorul sync, iar dacă nu-l anulează, ai un cluster care nu mai seamănă cu niciun commit. Diferența e că un agent poate face zece `kubectl apply` în treizeci de secunde, fără să obosească și fără să ezite.

De aceea gardul se pune **înainte** de prima unealtă, nu după prima pățanie. Ce are voie agentul, de azi încolo:

Are voie	Nu are voie
`kubectl get`, `describe`, `logs`, `top`	`apply`, `delete`, `edit`, `scale`, `patch`, `rollout`
`helm template`, `helm diff`	`helm install`, `upgrade`, `uninstall`
`argocd app get`, `diff`, `list`	`argocd app sync`, `set`, `delete`
`git status`, `log`, `diff`, `show`	`git commit`, `push`, `checkout`, `reset`
citit și scris fișiere în repo	trimis orice, oriunde

Ultimul rând e cel pe care oamenii îl uită și e cel mai important: **agentul propune o modificare în working tree, tu o citești, tu o comiți.** Un agent care poate comite singur e un agent care poate strica producția în timp ce tu te uiți pe altă fereastră.

Ce e altfel față de un chat

Până acum „AI" a însemnat probabil o fereastră în care lipeai un stack trace și primeai o bănuială. Claude Code e altceva:

Chat	Claude Code
tu lipești `values.yaml`	citește singur fișierele din folderul unde l-ai pornit
îți dă un `grep`, îl rulezi tu	rulează comanda și vede ce a ieșit
ține minte cât ține conversația	citește `CLAUDE.md` la fiecare pornire
îi explici de fiecare dată ce e `rsk`	uneltele tale devin comenzi `/nume`

Prima consecință practică: **contează enorm ce folder deschizi.** Pornit în `ms\`, vede toate cele cinci repo-uri deodată și poate răspunde la întrebări care taie prin ele. Pornit într-un singur repo, nu poate — și exact întrebările interesante la tine sunt cele care trec dintr-un repo în altul.

Pasul 1 — Atelierul

Deschide un terminal în folderul părinte

1

```
cd ms
claude
```

Prima întrebare, ca să vezi cu ochii tăi ce e diferit — pune-o exact așa, fără alt context:

```
Ce repo-uri sunt in folderul asta si ce face fiecare?
```

Gata când: ai citit răspunsul cu atenție. O să fie aproximativ corect și o să conțină cel puțin o afirmație greșită — pentru că a ghicit din nume de foldere. Ăsta e punctul de plecare al pasului 2.

Pasul 2 — `CLAUDE.md`, contextul care nu se mai repetă

Scrie harta platformei, o singură dată

2

Un fișier în rădăcina folderului `ms\`, citit automat la fiecare pornire. Aici scrii **o singură dată** ce altfel ai repeta în fiecare conversație. Scrie-l cu mâna ta, nu copiat de la mine — ăsta e exercițiul. Un punct de plecare:

`ms/CLAUDE.md`

```
# Platforma rsk

Cinci repo-uri, un cluster K3s, ArgoCD.

| Folder | Ce e |
|---|---|
| `data-service/` | Spring Boot, consumer Kafka, scrie în MySQL (MOCO, in-cluster) |
| `importer-service/` | Spring Boot, citește din MySQL și produce pe topicul `product-topic` |
| `client-microserv-vite/` | frontend React + Vite, auth prin Keycloak |
| `argo-ms-gitops/` | repo-ul de GitOps citit de ArgoCD |
| `ms-gitops/` | repo-ul de GitOps în care scrie CI-ul |

Ultimele două rânduri nu sunt o greșeală de scriere. Vezi `/lant`.

Namespace-uri: `business` (aplicații), `databases`, `messaging`, `monitoring`.
Realm-ul Keycloak e `rsk`. Domeniul public e `icode.mywire.org`.
Imaginile se publică pe Docker Hub sub `ion21/`.
Tag-ul de imagine = SHA-ul scurt al commit-ului din repo-ul serviciului.

Clusterul se citește, nu se modifică. Orice schimbare pleacă dintr-un commit.
```

Verificarea că e citit. Repornește Claude Code în `ms\` și întreabă exact atât, fără niciun alt context:

```
In ce namespace ruleaza data-service si de unde isi ia baza de date?
```

Gata când: răspunde `business` și MySQL-ul din cluster — atunci `CLAUDE.md` e citit. Dacă bâjbâie, ai pornit în alt folder: verifică cu `pwd`. Compară răspunsul cu cel de la pasul 1. Diferența dintre ele e tot ce ai de înțeles despre `CLAUDE.md`: nu l-ai făcut mai deștept, i-ai dat harta pe care tu o ai în cap.

Pasul 3 — Gardul

Interzici mutațiile în `settings.json` 3

```
mkdir -p .claude
```

`.claude/settings.json`

```
{
  "permissions": {
    "deny": [
      "Bash(kubectl apply:*)",
      "Bash(kubectl delete:*)",
      "Bash(kubectl edit:*)",
      "Bash(kubectl scale:*)",
      "Bash(kubectl patch:*)",
      "Bash(kubectl rollout:*)",
      "Bash(helm install:*)",
      "Bash(helm upgrade:*)",
      "Bash(helm uninstall:*)",
      "Bash(argocd app sync:*)",
      "Bash(argocd app set:*)",
      "Bash(argocd app delete:*)",
      "Bash(git commit:*)",
      "Bash(git push:*)",
      "Bash(git checkout:*)",
      "Bash(git reset:*)"
    ]
  }
}
```

Demonstrația. Cere-i agentului, direct:

```
Da scale la deployment-ul importer-service in namespace business la 2 replici.
```

Trebuie să refuze. Ăsta e momentul lecției: regula din prima pagină nu mai e o promisiune pe care ți-o fac eu, e ceva ce vezi pe ecran. Apoi, imediat, cere-i opusul:

```
Cate replici are acum importer-service in business si de cate ori a repornit?
```

Gata când: a răspuns. Ai obținut exact poziția din care vrei să lucrezi: **un agent care vede tot clusterul și nu poate schimba nimic în el.** Diagnostic fără risc.

Plasă, nu zid. Gardul se potrivește pe forma comenzii. Dacă agentul găsește altă cale către aceeași stricăciune — un `kubectl` chemat dintr-un script pe care tocmai l-a scris — regula nu se declanșează. Gardul e o plasă, nu un zid. Zidul rămâi tu, care citești ce rulează.

Pasul 4 — Prima unealtă, pe o întrebare pe care ți-ai pus-o deja

Până aici ai construit atelierul. Acum vezi la ce folosește, pe ceva ce te privește direct. Săptămâna trecută ai făcut o operație cu mâna: ai luat tag-ul unei imagini și l-ai scris într-un `values.yaml`. Dacă ai un pipeline care exact asta trebuie să facă, întrebarea care urmează e obligatorie: **de unde știi, în orice moment, dacă tag-ul pe care îl citește ArgoCD e același cu tag-ul pe care l-a scris ultima rulare de CI?**

Azi răspunzi la ea prin trei tab-uri de GitHub și un `argocd app get`. E o verificare pe care o faci cu ochii, deci o faci rar, deci o faci după ce s-a stricat ceva.

Întâi cu mâna, cu ceasul pornit — 4a

Înainte de orice îi ceri agentului: **verifică manual, cronometrat, pentru toate cele trei servicii**, dacă tag-ul din repo-ul citit de ArgoCD e același cu ce a publicat CI-ul ultima dată.

Gata când: ai notat minutele. Numărul ăsta e prima linie din jurnalul tău.

Apoi ceri — 4b

Cere în română, normal, și spune-i de la început **ce trebuie să iasă**, nu doar ce să facă:

```
Pentru fiecare serviciu (data-service, importer-service, client-microserv-vite):
gaseste tag-ul de imagine pe care il citeste ArgoCD si tag-ul scris de ultima
rulare de CI. Scoate un tabel cu serviciul, cele doua tag-uri, si OK sau DRIFT.
Daca un serviciu nu e deployat prin ArgoCD deloc, scrie asta explicit, nu-l sari.
Citeste doar fisierele si git log, fara sa modifici nimic.
```

Ce se întâmplă mai departe merită urmărit, pentru că aici e toată diferența față de un chat: **agentul deschide repo-urile, urmărește lanțul prin fișiere de workflow și values, și îți dă tabelul.** Nu-ți dă un `grep` pe care să-l rulezi tu.

Ultima propoziție din cerere — cea despre serviciul care nu e deployat — nu e acolo întâmplător. Uneltele scrise în grabă raportează despre ce găsesc și tac despre ce lipsește, iar tăcerea arată exact ca „totul e în regulă".

Poarta: o unealtă care nu poate spune „DRIFT" nu e o unealtă — 4c

Aici e partea pe care majoritatea o sar, și e singura care contează. Dacă unealta ți-a spus „OK peste tot", nu ai aflat că platforma e sănătoasă. Ai aflat că unealta a scris „OK". Sunt două lucruri diferite, și se despart exact atunci când te bazezi pe ea. Deci o forțezi să dea DRIFT, pe un caz în care tu știi sigur răspunsul:

- Într-un repo, schimbă cu mâna o cifră din tag-ul din `values.yaml`. Nu comiți nimic.
- Rulezi unealta din nou.
- **Trebuie** să iasă DRIFT, pe serviciul ăla, cu ambele tag-uri afișate.
- Pui cifra la loc. `git diff` gol.

Gata când: a ieșit DRIFT și `git diff` e gol. Dacă spune tot OK, unealta se uită în altă parte decât crezi tu, și tocmai ai evitat să te bazezi pe ea trei luni.

> **Reflexul cursului.** Ăsta e reflexul pe care îl exersăm tot cursul, și e același pe care îl ai deja când scrii un test: un test care trece pe cod stricat nu e un test. Aici nu e nimic nou, doar aplicat la unelte în loc de cod.

Transformă-l în comandă — 4d

O verificare pe care o faci la fiecare release nu se re-cere de fiecare dată. Cere-i agentului:

```
Transforma ce tocmai ai facut intr-o comanda /lant, cu verificarea inauntru:
sa nu raporteze niciodata doar OK, ci sa spuna si ce a comparat, fisier cu fisier.
```

Gata când: ai rulat `/lant` de două ori și ai primit aceleași tag-uri în ambele rulări = unealtă bună.

> **Ce înseamnă „același rezultat".** Modelul nu repetă niciodată aceleași propoziții, dar trebuie să repete aceleași **valori**. Dacă a doua oară sună altfel și arată la fel, e în regulă.

Și acum partea care contează

4e

Uită-te la ce-ți iese din `/lant` și răspunde-ți la o întrebare: **tabelul ăsta ar fi trebuit să existe înainte să fi pus tu tag-ul cu mâna, sau după?**

Asta e lecția, și n-are legătură cu AI-ul. Ai reparat o nepotrivire. Unealta îți arată de ce apare nepotrivirea. Sunt două lucruri diferite, iar al doilea e singurul care o oprește să revină luna viitoare.

Ce duci acasă

`/lant` rulat, și o decizie scrisă

1

Rulează unealta pe toate trei serviciile, salvează tabelul, și scrie sub el trei rânduri: care e cauza nepotrivirii, ce ai schimba ca să nu mai poată apărea, și de ce n-ai făcut-o încă. Al treilea rând e cel interesant — de obicei răspunsul e „nu știam că se poate" sau „știam, dar nu acum". Amândouă sunt răspunsuri bune. Ce nu e bun e să nu fi pus întrebarea.

> **Nu repara nimic până la lecția 2.** Reparația e capstone-ul, și vreau s-o faci cu unealta care îți confirmă că a mers, nu pe încredere.

Jurnalul — o săptămână

2

De fiecare dată când faci ceva repetitiv cu mâna pe platformă, un rând:

Ce am făcut	Unde	De câte ori / săpt.	Cât durează o dată	Ce mă enervează
verificat de ce nu sync-uiește o aplicație	ArgoCD UI + repo	3	15 min	mă uit în două repo-uri ca să răspund la o întrebare
căutat în ce namespace e un secret	kubectl	5	4 min	nu țin minte numele

Minimum 10 rânduri, iar la „cât durează" pui **minute măsurate**, nu estimate din cap. La final marchezi primele 3 după formula `de câte ori × cât durează`. Alea trei devin uneltele pe care le construim în lecțiile următoare.

Fără jurnalul ăsta, construim unelte pentru munca despre care eu presupun că o faci. Cu el, construim pentru cea pe care o faci într-adevăr.